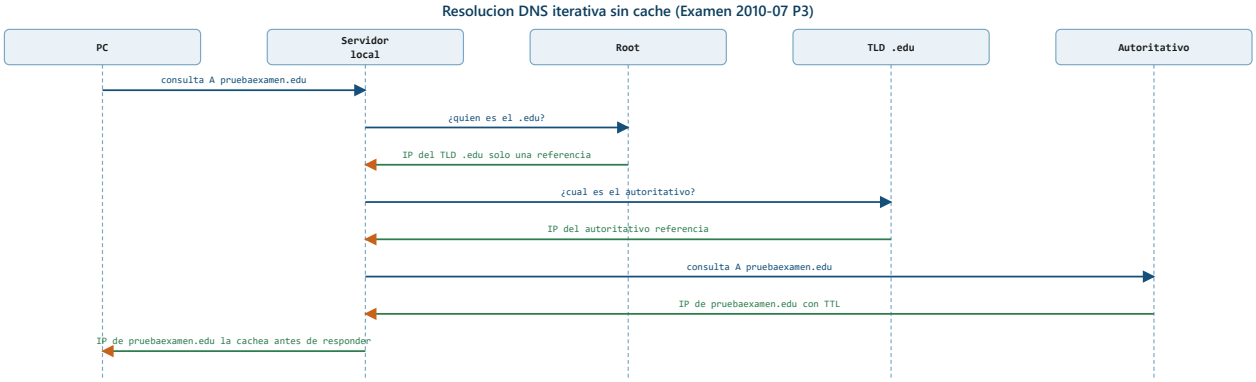
Clase 11 — DNS: iterativo vs. recursivo, caché y TTL



Bloque: B (Aplicación) · **Semana:** 4 · **Duración estimada:** 110 min **Correlativa:** Clase 10 (DNS: la base de datos distribuida) · **Sigue con:** Clase 12 (el resolver DNS en la práctica, con sockets) **Material de referencia:** Tanenbaum cap. 7 (§7.4.3, «Resolución de nombres») · RFC 1034, RFC 1035, RFC 6891, RFC 7766, RFC 8020, RFC 8484, RFC 7858, RFC 9250, RFC 8767, RFC 9460, RFC 4033-4035, RFC 5155

🎯 Al terminar esta clase vas a poder

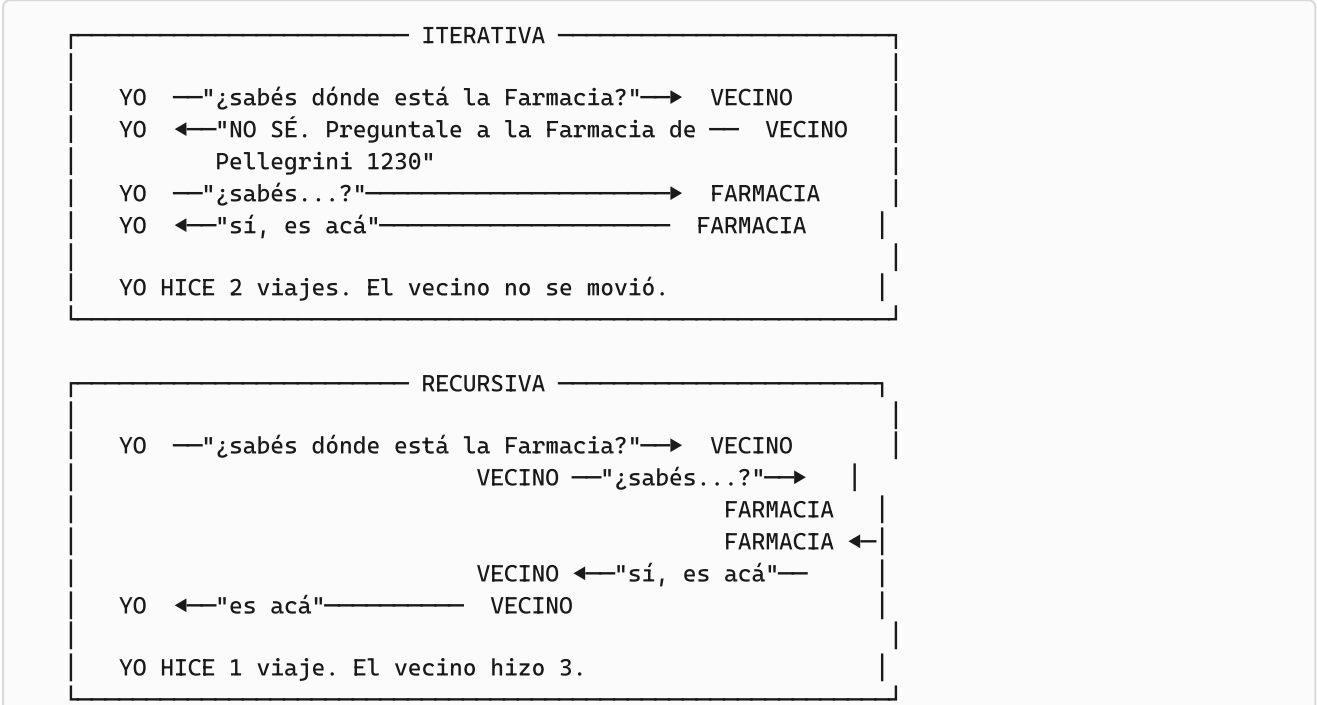
1. Explicar la diferencia entre una consulta **iterativa** y una **recursiva**, diciendo **quién se hace cargo** de seguir preguntando cuando el servidor consultado no tiene la respuesta.
2. Describir el **flujo real** de una resolución, desde el *stub resolver* del sistema operativo hasta el servidor autoritativo, y decir **qué parte es recursiva y qué parte es iterativa**.
3. Escribir los **mensajes DNS literales** de una resolución completa sin caché, y explicar **qué contesta cada uno** (incluida la diferencia entre una *referencia* y una *respuesta* final).
4. Explicar por qué existe la **caché**, por qué el TTL es un **compromiso**, **quién lo define** y **quién lo descuenta**.
5. Describir la **caché negativa**: qué es `NXDOMAIN`, qué es `SOA.MINIMUM`, por qué hay que cachear los errores y qué dice `RFC 8020`.
6. Resolver completo el **Examen 2010-07 Pregunta 3** (8 puntos) con su solución oficial a la vista.
7. Resolver completo el **Examen 2010-12 Pregunta 1** (8 puntos), los dos apartados, con su solución oficial a la vista.
8. Explicar qué cambió con **DoH**, **DoT**, **DoQ**, **DNSSEC** y **SVCB/HTTPS**, y por qué el punto 2 de arriba **no se enseña** en 2026.

> **Prerequisito del examen:** `2010-12 P1` (8 puntos — ★ el TTL del RR y los pasos de una consulta tipo A), `2010-07 P3` (8 puntos — ★ la resolución iterativa completa sin caché), `2010-02 P1b` (8 puntos — iterativa vs. recursiva), `2007-07 P4` (8 puntos — función del DNS y protocolo involucrado).

PARTE 1 — TEORÍA

1.1 Los dos algoritmos, definidos sin ambigüedad

El DNS tiene **dos** algoritmos de consulta, y la diferencia entre ellos se reduce a **una sola pregunta**: cuando el servidor al que consulté **no tiene** la respuesta, **¿quién sigue preguntando?**



	Iterativa	Recursiva
El que no sabe	devuelve una **referencia** («andá a preguntarle a este») y **no hace nada más**	**pregunta él** al nivel inferior, y así hasta obtener la respuesta final
Quién sigue preguntando	**el que preguntó** (el cliente)	**el que recibió la pregunta** (el servidor)
Qué vuelve	o una respuesta, o una referencia	**siempre** una respuesta final (o un error terminal)
Cuántos viajes ve el cliente	**muchos**: uno por nivel	**uno**: pide y recibe la respuesta
Quién la usa	el **resolver recursivo**, contra los autoritativos	el **host** (su *stub resolver*), contra el resolver local

Y el último punto es el que decide todo lo demás: si **cada host** de Internet tuviera que hacer los cuatro viajes él mismo, el root server tendría que atender miles de millones de consultas por segundo y todos los hosts estarían haciendo el mismo trabajo redundante. La **consulta recursiva** le delega ese trabajo a **un solo servidor**, que además es el único que puede **cachear** el resultado y devolverlo al siguiente cliente sin volver a bajar por la jerarquía.

> **La frase que hay que saber decir de memoria: la iterativa existe para que un solo servidor recursivo haga el trabajo pesado y cachee todo; la recursiva existe para que el cliente no tenga que hacerlo.**

Cómo se ve en el cable: los flags `RD` y `RA`

No hay que saberlo de memoria para el examen, pero **te da la certeza absoluta** de qué algoritmo se está usando:

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|            ID (16 bits)                                       |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|QR|   Opcode   |AA|TC|RD|RA|  Z  |AD|CD|     RCODE           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           QDCOUNT = número de preguntas                       |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           ANCOUNT = número de respuestas                      |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           NSCOUNT = número de registros de autoridad          |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           ARCOUNT = número de registros adicionales           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

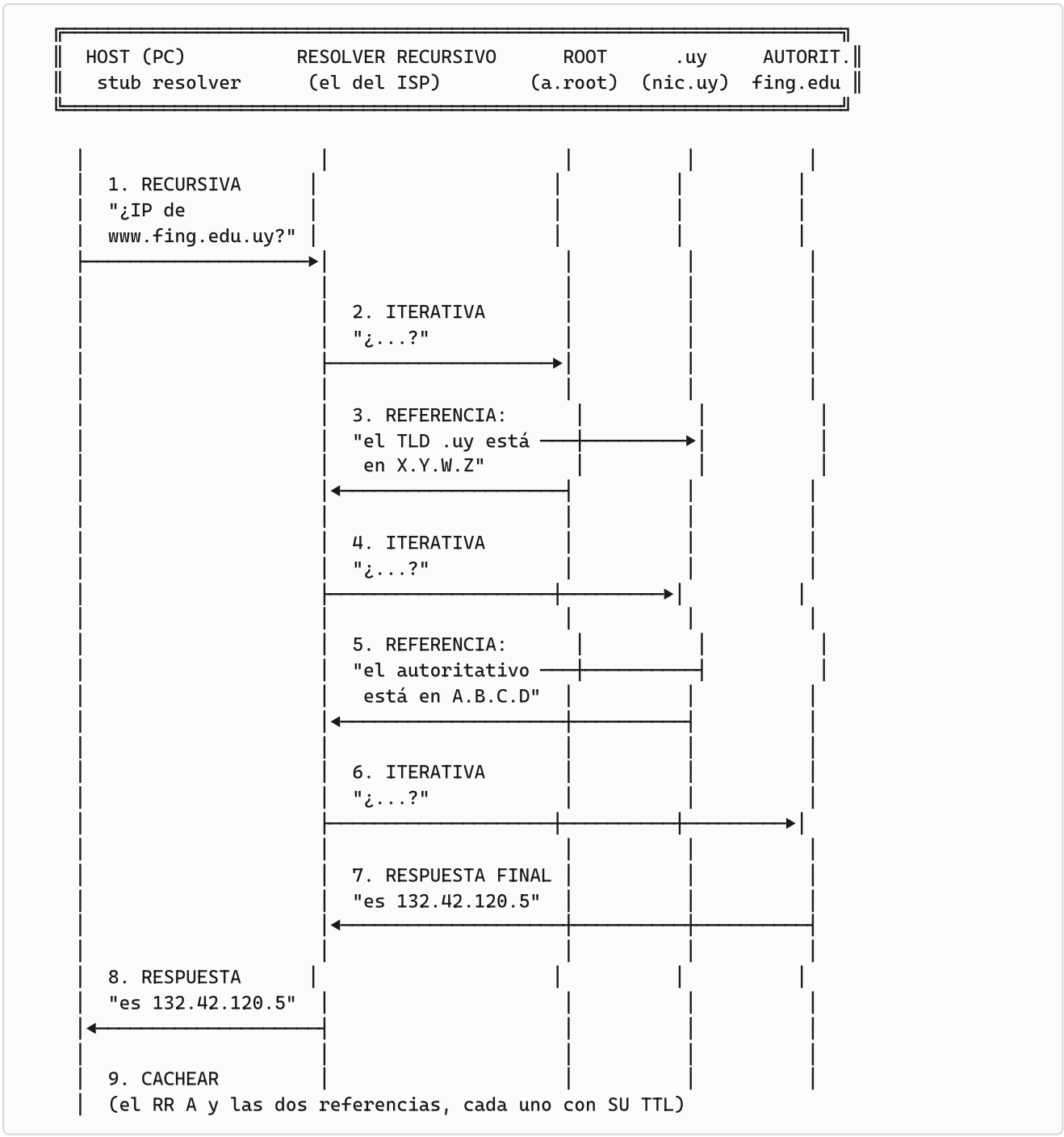
- `RD` (*Recursion Desired*, «quiero recursión»): lo pone **el que pregunta**. `RD = 1` significa «si no sabés, averiguatelo vos por mí».
- `RA` (*Recursion Available*, «recursión disponible»): lo pone **el que responde**, y dice si **ese servidor** está dispuesto a hacer de recursivo. Un **root server tiene `RA = 0` siempre**: los root no hacen de recursivos, jamás.
- `AA` (*Authoritative Answer*): lo pone el que responde y dice «esta es la respuesta final, la tengo yo». **Un root o un TLD contestan con `AA = 0`**, porque lo que están dando es una *referencia*, no una respuesta.
- `TC` (*Truncated*): la respuesta no entró y hay que repetir por TCP.
- `RCODE`: `0` NOERROR, `1` FORMERR, `2` SERVFAIL, `3` **NXDOMAIN**, `4` NOTIMP, `5` REFUSED.

Los dos valores que aparecen en los exchanges de esta clase:

```
Consulta del cliente:   QR=0  RD=1  →  flags = 0x0100   (1000 0000 0000 0000)
Respuesta de un root:   QR=1  RD=1  RA=0  →  flags = 0x8180   (REFERENCIA, no respuesta)
Respuesta del autorit.: QR=1  AA=1  RD=1  RA=0  →  flags = 0x8580   (RESPUESTA FINAL)
Respuesta del recursivo:QR=1  RD=1  RA=1  →  flags = 0x8180   (le devuelve el final al PC)
```

1.2 El flujo real: quién le pregunta a quién

Este es **el** diagrama de la unidad, y es la respuesta completa a `2010-02 P1b` y a `2010-07 P3`:



Lo que hay que leer en este diagrama:

1. **El mensaje 1 es RECURSIVO.** El *stub resolver* del sistema operativo le dice al resolver del ISP «resolveste esto por mí y traeme la respuesta final».
2. **Los mensajes 2 a 7 son ITERATIVOS.** El resolver recursivo es el que pregunta, y el que va bajando por la jerarquía es **él**, no su cliente.
3. **Los mensajes 3 y 5 NO son respuestas**: son **referencias** (*referrals*). Vienen con `ANCOUNT = 0` y el dato en la sección **Authority** + **Additional**.
4. **Solo el mensaje 7 es una respuesta final**, y es el único con `AA = 1`.
5. **El mensaje 8 no es un viaje nuevo**: es la vuelta del mensaje 1.

> ⚠️ **Esto no cambió desde 1987.** Ni la cadena de actores, ni los dos algoritmos, ni los puertos. Un parcial de FING de 2011 y uno de 2026 tienen **el mismo desenho de arriba**. Lo único que cambió es **quién ve el primer mensaje** (§1.7).

1.3 Los mensajes literales

Un mensaje DNS tiene **12 bytes de cabecera** y **cuatro secciones**. Todo lo que falta son **copias literales** de `dig`, que es como lo vas a ver en Wireshark.

La cabecera, byte a byte

```
ID = 0x3F2A        (lo elige el cliente; sirve para emparejar respuesta)
flags = 0x0100     (QR=0 consulta, RD=1 quiero recursión, RCODE=0)
QDCOUNT = 1        (una pregunta)
ANCOUNT = 0        (cero respuestas  ← OJO: todavía no hay respuesta)
NSCOUNT = 0        (cero registros de autoridad)
ARCOUNT = 0        (cero registros adicionales)
```

Mensaje 1 — el PC pregunta a su resolver local (petición)

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x3F2A
;; flags: rd; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;www.fing.edu.uy.               IN      A

;; MSG SIZE  rcvd: 0
```

Traducción campo por campo: `;; flags: rd` es el `RD = 1`. `QUESTION` es el `QDCOUNT`. Las tres secciones en cero son `ANCOUNT`, `NSCOUNT` y `ARCOUNT`. **`rd`** significa que el PC **quiere** recursión; que el servidor la conceda o no se ve en el `ra` de la respuesta.

Mensaje 3 — el root contesta: una referencia, no una respuesta

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x3F2B
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; QUESTION SECTION:
;www.fing.edu.uy.               IN      A

;; AUTHORITY SECTION:
uy.             172800  IN  NS   a.nic.uy.
uy.             172800  IN  NS   b.nic.uy.

;; ADDITIONAL SECTION:
a.nic.uy.       172800  IN  A    200.40.4.1
b.nic.uy.       172800  IN  A    200.40.4.2

;; MSG SIZE  rcvd: 132
```

Leé esto con cuidado, porque es el 80 % del examen de `2010-07 P3`:

```
   ANSWER: 0   ←  NO HAY RESPUESTA.  El root NO sabe la IP de www.fing.edu.uy
                  y NUNCA la va a saber: no es su trabajo.

   AUTHORITY: 1  +  ADDITIONAL: 1   ←  Esto es una DELEGACIÓN:
                  "el que sabe es a.nic.uy, y además, porque te va a servir,
                   te paso su dirección: 200.40.4.1"

   El flag `aa` NO aparece  →  AA = 0. Si fuera AA=1 el root estaría afirmandose
                               autoritativo sobre fing.edu.uy, lo cual es mentira.
```

Y el `ADDITIONAL SECTION` es exactamente el **glue record** de la Clase 10: el `.uy` no tiene por qué saber la IP de `a.nic.uy` (el nombre), pero **sí** tiene el `A` del servidor de nombres **por el que le delegó** `fing.edu.uy`, así que lo paga en el mismo mensaje y nos ahorra un viaje.

Mensaje 5 — el TLD `.uy` contesta: otra referencia

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x3F2C
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; QUESTION SECTION:
;www.fing.edu.uy.               IN      A

;; AUTHORITY SECTION:
fing.edu.uy.     172800  IN  NS   ns1.fing.edu.uy.
fing.edu.uy.     172800  IN  NS   ns2.fing.edu.uy.

;; ADDITIONAL SECTION:
ns1.fing.edu.uy. 172800  IN  A    132.42.60.1
ns2.fing.edu.uy. 172800  IN  A    132.42.60.2

;; MSG SIZE  rcvd: 140
```

Exactamente la misma forma que la del root, con `ANCOUNT = 0` otra vez. El `.uy` tampoco conoce los dominios: lo que tiene es el **`NS` de `fing.edu.uy`**.

Mensaje 7 — el autoritativo contesta: **la respuesta final**

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x3F2D
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;www.fing.edu.uy.               IN      A

;; ANSWER SECTION:
www.fing.edu.uy.   3600  IN  A   132.42.120.5

;; MSG SIZE  rcvd: 56
```

`flags: qr aa`: acá sí aparece el `aa` (*Authoritative Answer*). **Es la primera y única respuesta de la cadena** que tiene contenido. Y mirá el TTL: `3600`.

Mensaje 8 — el resolver recursivo le contesta al PC

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x3F2A
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;www.fing.edu.uy.               IN      A

;; ANSWER SECTION:
www.fing.edu.uy.   3600  IN  A   132.42.120.5

;; MSG SIZE  rcvd: 56
```

`flags: qr rd ra`: el `ra` (*Recursion Available*) confirma que el servidor **aceptó** hacer de recursivo. El `id` es **el mismo del mensaje 1** (`0x3F2A`): así el PC sabe que esta respuesta es la suya.

Resumen de los ocho mensajes

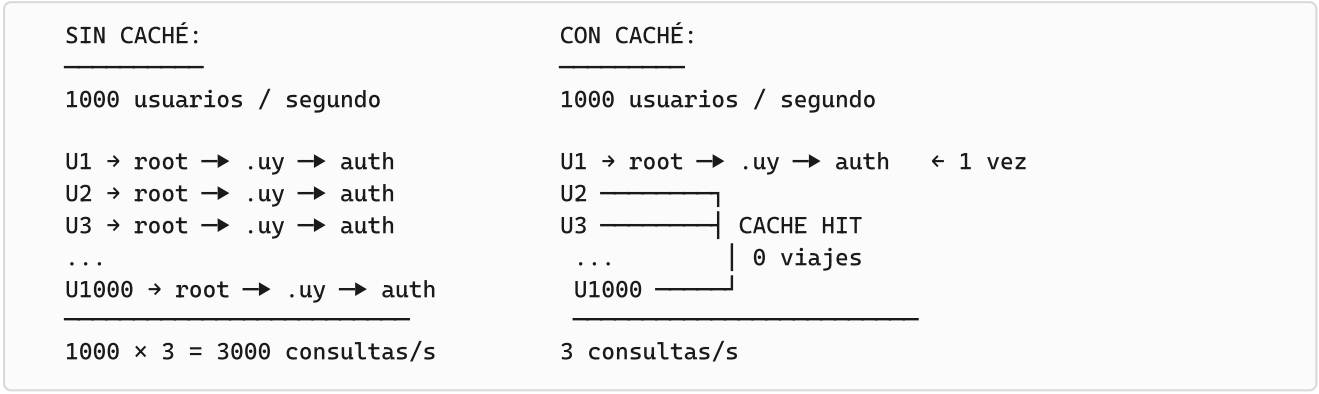
#	De	A	Tipo	ANCOUNT	AA	Qué transporta
1	PC (stub)	resolver local	**recursiva**	0	—	la pregunta
2	resolver local	root	iterativa	0	—	la pregunta
3	root	resolver local	**referencia**	**0**	0	`NS` de `.uy` + glue
4	resolver local	`.uy`	iterativa	0	—	la pregunta
5	`.uy`	resolver local	**referencia**	**0**	0	`NS` de `fing.edu.uy` + glue
6	resolver local	`ns1.fing.edu.uy`	iterativa	0	—	la pregunta
7	`ns1.fing.edu.uy`	resolver local	**respuesta**	**1**	**1**	el `A` con TTL 3600
8	resolver local	PC	respuesta	**1**	0	el mismo `A`

Cuatro ida y vuelta = 4 RTT. Uno solo trae la respuesta.

1.4 La caché, y por qué el TTL es un compromiso

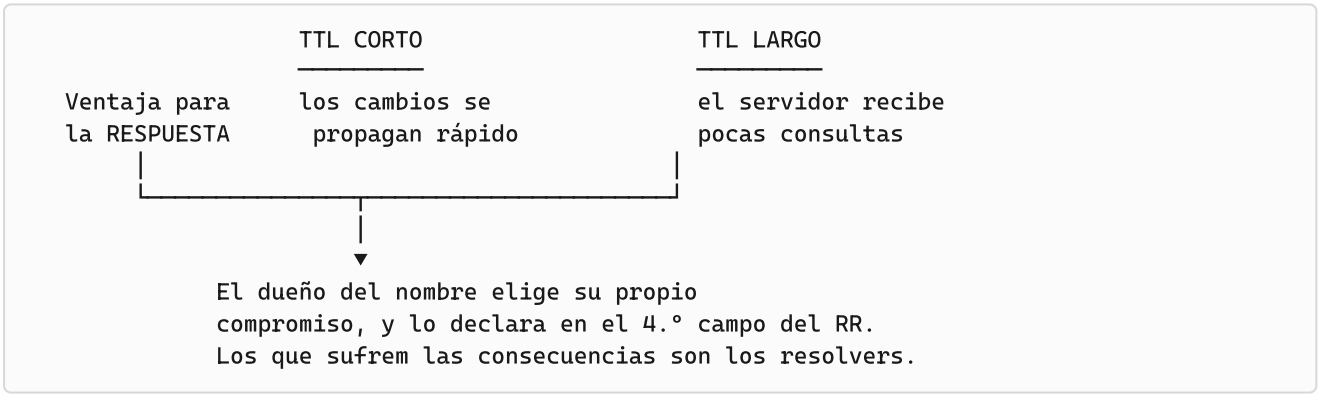
Por qué existe la caché

Sin caché, cada visita a una página generaría **2 a 6 consultas DNS**. Con `www.fing.edu.uy` como ejemplo, esto multiplicaría la carga sobre los 13 root exactamente como §1.1 explicó que **no puede ser**.



El compromiso del TTL

El TTL es un número que **elige el dueño del nombre**, y es exactamente eso: un compromiso entre **dos intereses opuestos** que **nadie** puede satisfacer a la vez.



Valor del TTL	Para qué se usa en la práctica	Ejemplo real
30 s a 5 min	contenido que cambia seguido: resultados de una búsqueda, el stock de una tienda, el resultado de una votación	un contador de visitas, un destino que cambia cada semana
300 s a 1 h	contenido dinámico pero no frenético: portadas de noticias	la home de un diario
3600 s a 86400 s	contenido estable: HTML, CSS, JS, imágenes	los assets de una aplicación
172800 s (2 días)	**los `NS` de una delegación**, por defecto	`fing.edu.uy. IN NS ns1.fing.edu.uy.`

> ⚠️ **El detalle fino que se pregunta:** el TTL de un registro `A` y el TTL del `NS` que delegó esa zona son **independientes**. El `NS` suele tener un TTL enorme (días) porque cambiar el servidor de nombres

> es raro; el `A` suele tener uno corto porque cambiar la IP de un servidor web es común. **Y el que va a doler es el `NS`**: si cambiás de servidor de nombres y no esperás a que venza su TTL, durante las horas siguientes una parte de Internet **sigue falando con el servidor viejo**. Es el mismo mecanismo del glue desactualizado que vimos en la Clase 10.

Los dos tiempos distintos que hay que separar: `max-age` y `TTL`

Esto confunde todo el tiempo y es pregunta clásica:

	`max-age` de HTTP	`TTL` de DNS
Dónde vive	cabecera `Cache-Control` de una respuesta HTTP	4.º campo de un Resource Record
Lo define	el **servidor web**	el **servidor autoritativo** de la zona
Unidades	segundos	**segundos** (siempre, en todas las versiones)
Medido desde	la `Date` de la respuesta	el momento en que **el resolver guardó** el registro
Quién lo descuenta	el que guarda (navegador o proxy)	**el que guarda**, o sea el resolver recursivo

La analogía que lo aclara: el `TTL` es como el **vence** de un producto en la alacena, y el `max-age` es el **vence** escrito por el cocinero en el plato. Los dos cuentan el mismo reloj, pero **el que lo pone es otro**, y por eso pueden no coincidir.

El reloj, con las cuentas a la vista

```
El autoritativo responde:            www.fing.edu.uy.  3600  IN  A  132.42.120.5
                                                       ▲
                                                       |  4 bytes = 3600 segundos
                                                       |  = 1 hora, menos 2400 s que ya
                                                       |    pasaron en la red (los TTL
                                                       |    arrancan a contar en la
                                                       |    respuesta, no en el origen)

t = 0 s        el resolver recibe y GUARDA:  TTL restante = 3600
t = 1200 s     alguien pregunta de nuevo:
               frescura restante = 3600 − 1200 = 2400 s   →  HIT, 0 bytes, 0 viajes
t = 3600 s     vence.  Freno, la entrada queda "stale" (vieja pero presente).
t = 3601 s     alguien pregunta:  fresco? NO  →  hay que REVALIDAR
t = 3650 s     llega la respuesta:  "sí, es 132.42.120.5, TTL 3600"
               → se RENUEVA: ahora vence en 3600 + 3650 = 7250 s

El campo `Age` que devuelve un proxy es justamente esto:
frescura restante = TTL del origen − Age
freshness_remaining = max-age − Age
```

El TTL es un contador decreciente en el resolver, no un reloj absoluto. Es la frase que hay que decir si te preguntan «¿a qué refiere el valor del TTL?».

1.5 El caché negativo: `NXDOMAIN` y `SOA.MINIMUM`

El problema

Un cliente escribe mal una URL: `www.fing.edu.uy/articulso/2026`. El navegador pregunta por el host, que **sí existe**, así que ese caso no es problema. El problema real es cuando el **nombre entero no existe**:

```
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 0x51C7
;; flags: qr aa rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 0

;; QUESTION SECTION:
;noexiste.fing.edu.uy.            IN      A

;; AUTHORITY SECTION:
fing.edu.uy.  3600  IN  SOA  ns1.fing.edu.uy. admin.fing.edu.uy. (
                             2026100601 ; serial
                             7200       ; refresh
                             3600       ; retry
                             1209600    ; expire
                             3600 )     ; minimum   ◀── el que importa acá
;; MSG SIZE  rcvd: 121
```

`NXDOMAIN` = el nombre no existe, no el tipo. Si el nombre existiera pero no tuviera un `A`, la respuesta sería `NOERROR` con `ANCOUNT = 0` (se llama **NODATA**). La diferencia importa:

```
www.fing.edu.uy.        A      → NOERROR, 1 respuesta      ✅
www.fing.edu.uy.        MX     → NOERROR, 0 respuestas     ⚠️ NODATA: existe, pero no es MX
noexiste.fing.edu.uy.   A      → NXDOMAIN, 0 respuestas    ❌ no existe el nombre
```

`SOA.MINIMUM`: el TTL del error

El campo `minimum` del `SOA` (el séptimo y último número) es **el TTL por defecto de las respuestas negativas**. Cuando un autoritativo responde `NXDOMAIN` sin poner un `SOA`, el resolver usa ese `minimum`. En la zona de ejemplo es `3600`.

```
El SOA también se cachea, y con SU propio TTL (el de la fila del SOA,
que acá es 3600). El `minimum` es el TTL por defecto del NXDOMAIN cuando
el autoritativo no manda un SOA explícito.

Ojo con la diferencia: el `minimum` NO es «duración del error»: es un
valor por defecto para cuando no hay otro disponible. El TTL que manda
es el del SOA que venga en la respuesta.
```

Por qué hay que cachear los errores

Contra lo que parece intuitivo, **cachear un `NXDOMAIN` es una protección**:

```
SIN CACHÉ NEGATIVA:
  1000 consultas por segundo de "www.escrito-mal.typo.uy"
    × 3 viajes (root → .uy → fing.edu.uy) = 3000 paquetes/s
    contra los servidores autoritativos, en perpetuo.
  Eso es un ataque de denegación de servicio MUY barato para el atacante:
  un solo PC genera 3000 paquetes/s contra una zona ajena.

CON CACHÉ NEGATIVA:
  el primer NXDOMAIN se cachea 3600 s.
  Los 999 siguientes se responden de la caché:
    0 viajes, 0 bytes, 0 carga para el autoritativo.
  El ataque se vuelve inofensivo.
```

Y hay un beneficio adicional: **el `NXDOMAIN` nunca debería convertirse en `A` por error tipográfico**, así que cachear el error durante un rato **filtra tráfico basura** de forma involuntaryaria.

`RFC 8020` (2018): NXDOMAIN es un error, no un nombre

⚠️ Acá está la norma que **corrige** un malentendido muy extendido:

```
Lo que la RFC PROHÍBE:
──────────────────────

Que un resolver trate un NXDOMAIN como si fuera un nombre VÁLIDO
y lo use para «inventar» respuestas para nombres parecidos.

El caso concreto (domain shadowing):
────────────────────────────────────

Se consulta:      victima.example.com     →  NXDOMAIN
Se cachea:        (victima.example.com → NO EXISTE)

Si un atacante logra que en example.com quede un comodín (*.example.com)
o si más tarde el dominio aparece, un resolver mal implementado puede
seguir sirviendo el NXDOMAIN cacheado y «tapar» un nombre que SÍ existe.
Eso es DOMAIN SHADOWING: un nombre legítimo, sombreado por su ausencia.

Lo que la RFC RECOMIENDA:
─────────────────────────

1. Cachear el NXDOMAIN (sí, se cachea: protege contra el ataque de
   arriba).
2. Guardar la ENTRADA NEGATIVA CON LA CLAVE DEL NOMBRE EXACTO.
   Un NXDOMAIN de victima.example.com NO habilita a responder
   NXDOMAIN de otra.example.com: cada nombre negativo tiene su
   propia entrada y su propio timer.
3. Tratarlo como «este nombre no existe en este momento», no como
   «este nombre no existe para siempre».
```

La contradicción aparente que hay que saber explicar: cachear **sí**, no contaminar **no**. Se cachea **por nombre exacto** y con su propio timer, de modo que el error no se derrame sobre los nombres hermanos.

1.6 ⚠️ Actualización a 2026 — lo que cambió y lo que no

Sigue igual — de memoria, sin dudar:

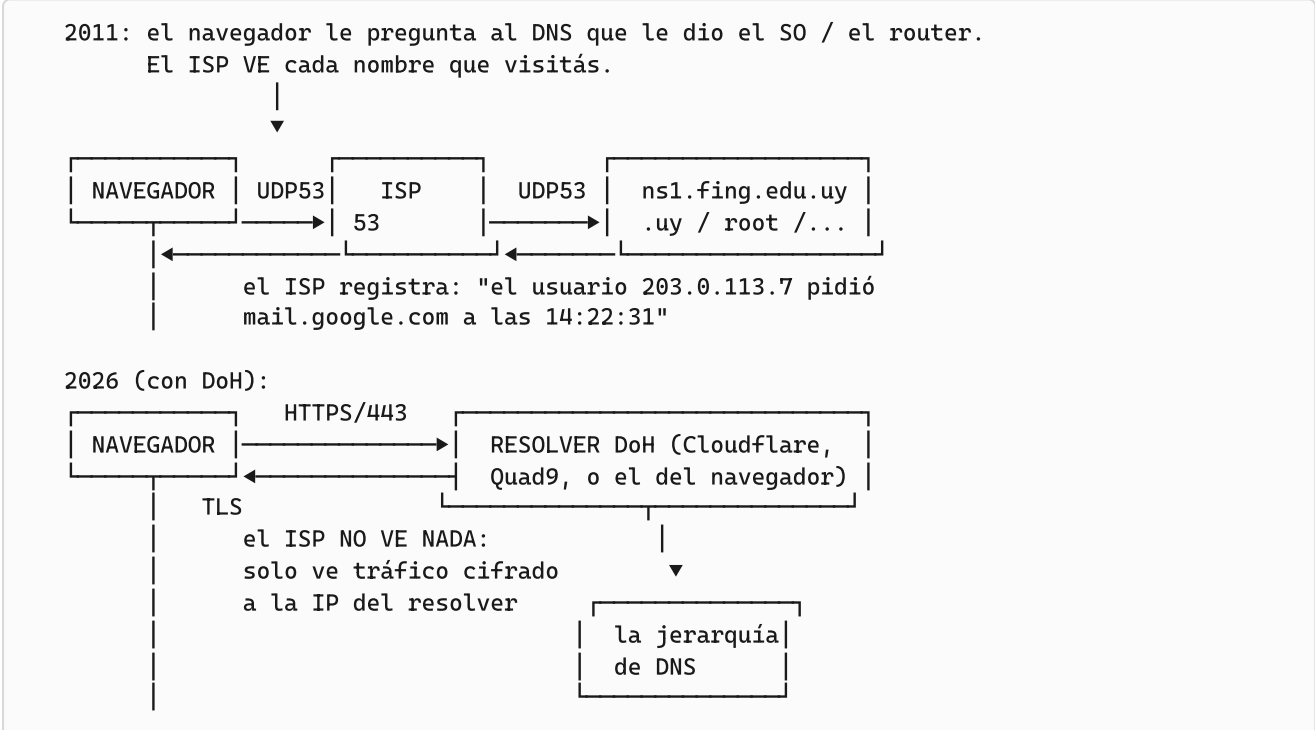
- **Jerarquía y delegación**: root → TLD → autoritativo, con `NS` y *glue records*.

- **Iterativa vs. recursiva**: el *stub resolver* pregunta **recursivamente** al resolver local; el resolver local pregunta **iterativamente** a root, TLD y autoritativo. **No cambió desde 1987.**
- **Estructura del mensaje**: ID, *flags* (`QR`, `Opcode`, `AA`, `TC`, `RD`, `RA`, `RCODE`), `QDCOUNT` / `ANCOUNT` / `NSCOUNT` / `ARCOUNT`, y las cuatro secciones.
- `RCODE`: NOERROR, FORMERR, SERVFAIL, **NXDOMAIN**, NOTIMP, REFUSED.
- **Tipos de registro**: `A`, `AAAA`, `NS`, `CNAME`, `MX`, `SOA` (con sus cinco números), `PTR`, `TXT`, `SRV`, `CAA`.
- `TTL` **y caché.** El TTL sigue siendo el cuarto campo, en segundos, definido por el autoritativo y descontado por quien cachea.
- **Puertos**: **53 UDP y 53 TCP**, y **853** (DoT, `domain-s`).
- **Round-robin** por varios registros `A`. Chrome y Safari lo aleatorizan por privacidad, y por eso las respuestas no siempre vienen en el mismo orden.

Cambió — y acá hay que ser preciso:

(a) DoH (`RFC 8484`, 2018): el cambio mayor

DNS over HTTPS es DNS **encapsulado dentro del protocolo HTTP**, sobre TLS, en el puerto **443**. El efecto es brutal y hay que saber expresarlo bien:



> ⚠️ **La conclusión que hay que escribir en el examen: con DoH, el DNS del navegador ya no lo ve tu ISP.** Pasa de estar en **UDP 53 plano** —donde todo el camino puede ver y alterar la pregunta— a estar **dentro de un HTTPS**, que es lo mismo que cualquier visita a una web. Y un HTTPS es **la una cosa que el ISP no puede leer sin romper el TLS**.

Lo que DoH cambia de verdad, en tres puntos:

1. **Confidencialidad**: el ISP no ve qué nombres se consultan (sí ve a qué IPs se conectan, salvo con ECH; eso es otra historia).
2. **Integridad**: nadie en el camino puede más inyectar una respuesta falsa (`DNS poisoning` o *pharming*).
3. **El salto a la red sigue siendo UDP 53**: DoH solo cifra el **tramo navegador → resolver**. Las consultas del resolver a root, TLD y autoritativo **siguen en DNS plano** salvo que ese resolver también hable DoT o DoQ.

(b) Chrome NO cambia tu resolver: el matiz que hay que saber

Esto se dice mal en todos lados y es un error de examen fácil de cometer:

```
LO QUE SE DICE (FALSO):
"Chrome usa DNS de Google por defecto"

LO QUE PASA (VERDAD):
┌──────────────────────────────────────────────────────────────┐
│ Chrome 83 (mayo-2020) NO cambia tu resolver.                 │
│ Intenta "subir" a DoH TU resolver actual, y lo hace SOLO si  │
│ ese resolver está en su LISTA DE PROVEEDORES CONOCIDOS.      │
│ Si no está en la lista → DNS plano, como siempre.            │
│ Si el usuario o el administrador lo configuran a mano →      │
│   puede apuntar a donde quiera.                              │
└──────────────────────────────────────────────────────────────┘
Chrome 87 (nov-2020) es el que trae HTTP/3 por defecto; no lo confundas
con el rollout de DoH.
```

> ⚠️ **Si te preguntan «¿el navegador usa el DNS de Google?», la respuesta es «no: Chrome intenta cifrar la consulta con el resolver que ya tenías, y solo si ese resolver está en su lista de proveedores conocidos; si no, sigue en DNS plano».**

(c) DoT (`RFC 7858`, 2016) y DoQ (`RFC 9250`, 2022)

- **DoT** (*DNS over TLS*): DNS cifrado sobre TLS, en el **puerto 853**, con nombre fijo (`domain-s`). Es lo que usan los **resolvers** (no el navegador) y lo que configurás a mano en un servidor.
- **DoQ** (*DNS over QUIC*): DNS cifrado sobre **QUIC**, en el **puerto 853**, usando el `Connection ID` y los streams independientes de QUIC. **DoQ elimina el *head-of-line blocking* de TCP**: si un paquete de una consulta se pierde, no frena a las demás consultas que van en la misma conexión.

```
DoT  = DNS dentro de TLS dentro de TCP       (ordenado, un stream)
DoQ  = DNS dentro de QUIC                   (multiplexado, N streams)
DoH  = DNS dentro de HTTP/2 o HTTP/3        (multiplexado, N streams)
──────────────────────────────────────────────────────────────────
Los tres cifran. Solo DoQ y DoH se benefician de la multiplexación.
```

(d) DNSSEC: estandarizado en 2005, despliegue bajo en 2026

La cadena de confianza de DNSSEC está **operativa**: `RFC 4033-4035` (2005) y `RFC 5155` (raíz firmada, 2010). Los números de despliegue son los que hay que saber:

Métrica	Valor
gTLD firmados	**~99,6 %**
Dominios (2LD) con `DS`	**~5-7 %**

> ⚠️ **La lección, literal:** DNSSEC es el ejemplo perfecto de **«estandarizado en 2005 y con despliegue bajo casi veinte años después»**. El eslabón fuerte (TLD) está casi completo; el eslabón débil (el dominio concreto) casi no lo tomó. Por eso en 2026 **no se enseña que «el DNS es seguro»**, sino que **tiene una capa de autenticidad opcional, bien diseñada y masivamente subutilizada**. Compará con HTTPS: mismo problema, misma forma, mismos veinte años.

`RFC 8080` agrega EdDSA (Ed25519) como algoritmo de firma y `RFC 8901` permite **firmantes múltiples** (*multi-signer*), que es lo que permite una migración de algoritmo sin cortar el servicio.

(e) `RFC 8767` (2020): servir caché vencida (*stale*)

Qué cambió en la semántica de caché. Antes, un registro vencido **se botaba**. **descartado**. Con `RFC 8767`, si el servidor autoritativo **no responde**, el resolver puede **servir la copia vieja**, marcándola como *stale*.

```
ANTES:  TTL vencido + autoritativo caído  →  SERVFAIL, el usuario ve un error.
AHORA:  TTL vencido + autoritativo caído  →  se sirve la copia vieja con un
                                               aviso de que está vencida.

¿POR QUÉ? Porque es resiliencia frente a DDoS: si alguien tumba un
autoritativo, los resolvers con stale siguen sirviendo en vez de caer
todos juntos. Es un compromiso explícito entre «frescura» y «disponibilidad».
```

El riesgo es evidente y hay que decirlo: **el contenido puede estar viejo**. Es un cambio de diseño real en el modelo de caché, no un detalle.

(f) `SVCB` / `HTTPS` (`RFC 9460`, 2023): el DNS transporta política de transporte

Un registro `HTTPS` (que es un `SVCB` en modo 1) dice **qué versiones de HTTP soporta el servidor, en qué IP está, con qué prioridad** — todo **antes** de abrir ningún socket:

```
;; QUESTION SECTION:
;www.fing.edu.uy.               HTTPS

;; ANSWER SECTION:
www.fing.edu.uy.   300  IN  HTTPS 1 . alpn="h3,h2" ipv4hint=132.42.120.5 port=443
```

Por qué importa: es lo que permite **descubrir HTTP/3 antes de conectar**, sin necesidad de haber hecho antes un viaje por HTTP/1.1 para enterarte por `Alt-Svc` (Clase 9). Es **el mismo cambio conceptual que `CAA`**: el DNS dejó de ser solo un directorio de direcciones y pasó a **transportar política** sobre el dominio. `RFC 9462` (DDR) permite descubrir resolvers en `_dns.resolver.arpa` sin configurar nada.

(g) EDNS(0) (`RFC 6891`) y conexiones persistentes (`RFC 7766`)

- `RFC 6891` (2013, *Internet Standard*): agrega el pseudo-registro `OPT`. Sin él, el máximo sigue siendo **512 bytes** y las respuestas grandes se cortan con `TC = 1`. Con él, el cliente anuncia `udp: 4096` y se evita el corte.
- `RFC 7766` (2016): los resolvers **deben mantener las conexiones TCP abiertas y reutilizarlas**; tienen prohibido cerrar la conexión por transacción. Si no, cada consulta por TCP pagaría un handshake.

> **Consecuencia para el examen:** si te preguntan *«¿qué cambió en el DNS?»*, la respuesta es: **nada de la arquitectura**. Ni la jerarquía, ni la delegación, ni el `RR` de cinco campos, ni los dos algoritmos cambiaron desde 1987. Lo que se **agregó** es cifrado (DoH/DoT/DoQ), autenticidad (DNSSEC, subdesplegada), extensiones (EDNS(0)), semántica de caché resiliente (`RFC 8767`) y política de transporte (`SVCB` / `HTTPS`).

1.7 El transporte: UDP, TCP y por qué DoH le ganó a DoT

El DNS normal viaja sobre **UDP 53**, y eso no es un capricho: una consulta es un datagrama chico (menos de 100 bytes) que se manda una sola vez, sin garantía de llegada, y **el reintento lo hace el resolver** si no recibe nada.

Situación	Transporte
Consulta normal	**UDP 53**
La respuesta no entra (banderita `TC = 1`)	se repite por **TCP 53**
Transferencia de zona (`AXFR`, `IXFR`)	**TCP 53**, siempre
DoT	**TCP 853**
DoQ	**UDP 853**
DoH	**TCP o UDP 443** (el mismo 443 de la Web)

```
El 443 compartido es la razón por la que DoH le ganó a DoT:

DoT necesita el puerto 853 abierto. En muchas redes corporativas,
UROs empresas y universidades el 853 está BLOQUEADO, y el resolver
del ISP a veces ni lo habla.

DoH usa el 443, que es EL puerto de la Web: si podés abrir una
página, podés resolver un nombre. Por eso DoH es el que se desplegó
en el navegador, y DoT se quedó en el mundo de los resolvers.
```

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — La cadena completa con caché en el TLD

Enunciado. Un PC en Montevideo resuelve `www.fing.edu.uy`. El resolver recursivo tiene **el `NS` de `fing.edu.uy` cacheado** (con su *glue*), pero **no** tiene el registro `A`. RTT entre el PC y el resolver: **2 ms**; entre el resolver y el root: **40 ms**; entre el resolver y el `.uy`: **25 ms**; entre el resolver y el autoritativo: **15 ms**. Todos los mensajes viajan por UDP, así que cada consulta cuesta **un RTT** y nada más.

a) ¿Cuántos mensajes y cuántos RTT?

```
El resolver tiene el NS de fing.edu.uy cacheado → SE SALTA el root y el .uy.
O sea que el mensaje 2 (al root) y el 5 (al .uy) NO existen.

┌─────┬──────────────────────┬──────────┬─────────┬──────────────────────────┐
│  #  │ De                   │ A        │ RTT     │ Tipo                     │
├─────┼──────────────────────┼──────────┼─────────┼──────────────────────────┤
│  1  │ PC (stub)            │ resolver │  2 ms   │ RECURSIVA (pregunta)     │
│  2  │ resolver             │ ns1.fing │ 15 ms   │ ITERATIVA (pregunta)     │
│  3  │ ns1.fing.edu.uy      │ resolver │ ↑ el mismo RTT                     │
│  4  │ resolver             │ PC       │ ↑ el mismo RTT                     │
└─────┴──────────────────────┴──────────┴─────────┴──────────────────────────┘

TOTAL: 4 mensajes, 2 RTT

Cálculo:
RTT_total = RTT(PC ↔ resolver) + RTT(resolver ↔ autoritativo)
          = 2 ms + 15 ms
          = 17 ms
```

Comparación con el caso sin caché (los 4 viajes del §1.3):

```
SIN NINGUNA CACHÉ:                CON CACHÉ DEL NS EN EL TLD:
──────────────────                ──────────────────────────

8 mensajes, 4 RTT                 4 mensajes, 2 RTT

2 + 40 + 25 + 15 = 82 ms          2 + 15 = 17 ms

Ahorro: 65 ms (79 %)              ┌──────────────────────────────────┐
                                  │ La caché no guarda la RESPUESTA,│
                                  │ guarda la DELEGACIÓN. Y la      │
                                  │ delegación es el 90 % del trabajo.│
                                  └──────────────────────────────────┘
```

b) ¿Cuántos RTT si además el `A` estuviera cacheado?

```
Si el registro A está en caché y NO venció:
  el resolver responde al PC sin salir a la red externa.

  ┌─────┬──────────┬──────────┬─────────┬──────────────────────────────┐
  │ 1   │ PC       │ resolver │ 2 ms    │ RECURSIVA                    │
  │ 2   │ resolver │ PC       │ ↑       │ respuesta desde la caché     │
  └─────┴──────────┴──────────┴─────────┴──────────────────────────────┘
  TOTAL: 2 mensajes, 1 RTT = 2 ms

Si el registro A está en caché pero VENCIÓ (hay que revalidar):
  el resolver tiene que ir al autoritativo de nuevo.
  RTT_total = 2 ms (PC) + 15 ms (autoritativo) = 17 ms
```

c) ¿Y si el PC tiene el nombre en su propia caché local?

```
Windows, macOS y Linux mantienen una caché de nombres en el cliente
(y el navegador tiene la suya). Si el nombre está fresco ahí:

  RTT = 0.  Ni siquiera sale un paquete a la red.

Esto es lo que hace que «abrir la misma pestaña dos veces» sea instantáneo.
```

d) La tabla que resume los cuatro escenarios:

Qué tiene cacheado el resolver	Mensajes	RTT	Tiempo (con los RTT del enunciado)
Nada (ni el `NS` ni el `A`)	8	4	2+40+25+15 = **82 ms**
El `NS` de `fing.edu.uy` (caso del enunciado)	4	2	2+15 = **17 ms**
El `A`, vencido	4	2	2+15 = **17 ms**
El `A`, fresco	2	1	**2 ms**
El `A`, fresco en la caché del PC	0	0	**0 ms**

Interpretación: **la caché ataca el tiempo, no los bytes.** Notá que en el primer caso y en el segundo se transfieren **los mismos 56 bytes** de respuesta final; lo que cambia es cuántos viajes hay que hacer para CONSEGUIRLOS.

⚠️ Errores típicos

- **Contar 8 RTT en vez de 4.** Los 8 mensajes son 4 parejas ida y vuelta. Cada mensaje suelto no es un RTT.
- **Contar el mensaje 8 como un viaje nuevo.** El mensaje 8 es la vuelta del 1.
- **Creer que el TTL del `NS` y el del `A` son el mismo número.** Son campos distintos, y el `NS` suele tener un TTL mucho más grande.
- **Olvidar que la caché del TLD se guarda con su propio TTL** (típicamente 172800 s = 2 días), así que no sirve para el caso «lo acabo de cambiar».

2.2 Ejercicio 2 — ¿Cuánto dura una entrada de caché? Con las cuentas

Enunciado. El autoritativo de `fing.edu.uy` responde:

```
www.fing.edu.uy.   3600  IN  A   132.42.120.5
fing.edu.uy.     172800  IN  NS  ns1.fing.edu.uy.
```

El resolver guarda la respuesta **a las 10:00:00**. Hacele estas preguntas: **(a)** ¿cuánto tiempo se puede usar sin revalidar? **(b)** ¿qué pasa exactamente a las 11:00:00? **(c)** ¿hasta cuándo vive la entrada del `NS`? **(d)** ¿cuántos viajes ahorra la caché en las **primeras 24 horas** si se pide el nombre **cada 10 segundos**?

a) ¿Cuánto se puede usar sin revalidar?

```
TTL = 3600 segundos = 1 hora.

Frescura restante = 3600 − 0 = 3600 s a las 10:00:00.

t = 10:00:00 ──────────────────────────────── t = 11:00:00
    se guarda                                  vence
    (3600 s de frescura)                       (0 s de frescura)
```

b) Interpretación: entre las 10:00 y las 11:00, **cualquier** consulta se responde de la caché: **0 mensajes, 0 bytes, 0 RTT**.

b) ¿Qué pasa exactamente a las 11:00:00?

```
Las 11:00:00 es el instante límite. La respuesta es:

┌──────────────────────────────────────────────────────────┐
│ • La entrada NO se borra. Pasa de "fresca" a "VENCIDA"   │
│   (en inglés, stale: vieja pero presente).               │
│                                                          │
│ • La SIGUIENTE consulta ya NO es un acierto de caché:    │
│   el resolver tiene que revalidar.                       │
│                                                          │
│ • Si el servidor contesta, el resolver reemplaza la entrada:
│     - si dice "no cambió" (el RR es el mismo, byte a byte),
│       RENUEVA 3600 s más.                                │
│     - si dice "cambió", reemplaza el contenido y el TTL. │
│                                                          │
│ • Si el servidor NO contesta, hay dos salidas:           │
│     - servir la copia vieja (lo que autoriza RFC 8767)   │
│     - o devolver SERVFAIL (el comportamiento clásico)    │
│                                                          │
│ • Recién AHORA, si pasó lo mismo (vencimiento + revalidación
│   + no contestó), la entrada se elimina de verdad.       │
└──────────────────────────────────────────────────────────┘
```

El error conceptual número uno: decir que «a los 3600 segundos se borra del caché». No: a los 3600 segundos deja de ser *fresco*. Borrarse es otra etapa, y la distinción importa justamente porque `RFC 8767` permite usar esa copia vieja.

c) ¿Hasta cuándo vive la entrada del `NS`?

```
TTL del NS = 172800 s = 48 horas = 2 días.

Se guardó a las 10:00:00 del lunes.
Vence:      10:00:00 del MIÉRCOLES.

t = lun 10:00 ──────────────────────────────────────► mié 10:00
                  172800 s = 48 h = 2 días
```

Interpretación: el `NS` vive **48 veces más** que el `A`. Es correcto, porque cambiar el **servidor de nombres** es rarísimo y cambiar la **IP de un servidor web** es de todos los días. Y por eso, si vos cambiás de servidor de nombres, **tenés que esperar dos días** (o bajar el TTL del `NS` con antelación) antes de tocar el glue que ya está en la zona del TLD.

d) ¿Cuántos viajes ahorra la caché en 24 horas, si me piden el nombre cada 10 s?

```
Consultas en 24 horas:
N = 86 400 s / 10 s = 8 640 consultas

Sin caché:
  8 640 × 4 RTT = 34 560 viajes  (y 8 640 × 8 = 69 120 mensajes)

Con caché (TTL 3600 s = 1 hora):
  revalidaciones = 24 h / 1 h = 24
  aciertos de caché = 8 640 − 24 = 8 616

  Viajes = 24 revalidaciones × 4 RTT = 96 RTT
           + 0 RTT de las 8 616 consultas acertadas
           = 96 viajes

AHORRO:
  viajes:     34 560 − 96 = 34 464 viajes  →  99,72 % menos
  mensajes:   69 120 − 192 = 68 928        →  99,72 % menos
```

	Consultas	RTT totales	Mensajes	Carga en el autoritativo
Sin caché	8 640	34 560	69 120	8 640 consultas
Con caché TTL=3600	8 640	**96**	192	**24** consultas
Ahorro	—	**99,72 %**	**99,72 %**	**99,72 %**

Interpretación: **por eso el DNS sobrevive.** Si cada cliente de Internet repitiera los cuatro viajes, los 13 root servers no darían abasto. El TTL es literalmente lo que hace posible la escala del sistema. **Y el precio es la propagación lenta**: un cambio tarda hasta un TTL en ser visto por todo el mundo.

> **La frase del examen:** *el TTL es un compromiso entre la corrección de los cambios y la carga de la red; lo elige el servidor autoritativo, lo descuenta quien cachea, y por eso es siempre un compromiso, nunca una solución.*

⚠️ Errores típicos

- **Decir que el TTL se descuenta en el autoritativo.** No: el autoritativo solo lo **pone**. Quien lo descuenta es el **resolver que lo guardó**.

- **Confundir «vence» con «se borra».** Vencer es dejar de ser fresco; borrar es el paso siguiente, y a veces no ocurre.
- **Contar el TTL en minutos.** Siempre son **segundos**.
- **Asumir que el TTL del `NS` es el del `A`.** No tienen por qué ser iguales, y casi nunca lo son.

2.3 Ejercicio 3 — Por qué `NXDOMAIN` se cachea, y qué dice `RFC 8020`

Enunciado. Un atacante con un solo portátil quiere tumbar el servidor de nombres de `fing.edu.uy`. Su script hace 5 000 consultas por segundo de nombres inexistentes en esa zona, por ejemplo `a1.fing.edu.uy`, `a2.fing.edu.uy`, …, con `TTL` de caché negativa de `SOA.MINIMUM = 3600`. **a)** ¿Con caché negativa, funciona el ataque? **b)** ¿Y si los resolvers de los usuarios **no** cachean los negativos? **c)** ¿Qué dice `RFC 8020` y por qué «cachear sí, contaminar no» no es contradictorio?

a) Con caché negativa, ¿funciona?

```
Nombres atacados:  a1.fing.edu.uy, a2.fing.edu.uy, a3...  (5000 distintos/s)

Sin caché negativa:
  5000 consultas/s × 3 viajes (root → .uy → fing.edu.uy)
  = 15 000 paquetes/s contra los servidores de nombres de fing.edu.uy
 Estimado: 15 000 × ~500 bytes = 7,5 MB/s sostenidos.
  Con un solo portátil eso es nada. Y es 5000 consultas/s de CPU
  de resolución de zona.

CON caché negativa (SOA.MINIMUM = 3600):
  Cada nombre DISTINTO genera un NXDOMAIN y se cachea 1 hora.
  Los 5 000 nombres/s siguen generando entradas nuevas... pero:

  ⚠️ OJO: si el atacante usa 5 000 nombres DISTINTOS por segundo,
     la caché negativa los guarda a todos, y la tabla crece sin
     control. La caché negativa protege del ATAQUE DE REPETIR EL
     MISMO NOMBRE, no del ataque de inundar la tabla con nombres
     siempre nuevos.

El ataque que la caché negativa SÍ detiene:
  5000 consultas/s del MISMO nombre (por ejemplo "noexiste")
    → 1 NXDOMAIN, cacheado 3600 s
    → las 4999 restantes se responden de la caché: 0 viajes.
  ATAQUE NEUTRALIZADO al 99,98 %.
```

Interpretación: la caché negativa es una protección real **contra la repetición**, y por eso `RFC 8020` la recomienda explícitamente. Lo que **no** hace es defenderse de un atacante que genera nombres aleatorios: para eso hacen falta los límites de tasa (*rate limiting*) del lado del atacante (o sea, de tu servidor de nombres).

b) ¿Y si los resolvers no cachean los negativos?

```
El estado se vuelve una máquina de estados:

t=0     atacante empieza          ┌──────────────┐
...     5000 consultas/s          │  RESOLVER    │  ← 15 000 paquetes/s salen
t=60s   el autoritativo           │  (el del ISP) │     hacia la zona de la
       empieza a caer:            │              │     víctima
       CPU al 100 %               └──────┬───────┘
              │                          │
              │    "gRPC: overload"      │
              ▼                          ▼
       la resolución de                LA VÍCTIMA
       fing.edu.uy SE CAE               SE CAE
              │
              ▼
       Y AHORA: los usuarios LEGÍTIMOS
       no pueden resolver NADA de
       fing.edu.uy  ←  esto es DoS real

Con caché negativa: los usuarios legítimos siguen resolviendo
desde la caché durante 1 hora. EL DOMINIO NO SE CAE.
```

Interpretación: **la caché negativa es lo que separa un ataque de una denegación de servicio real.** Con ella, el atacante gasta recursos del atacante; sin ella, gasta recursos de la víctima.

c) ¿Qué dice `RFC 8020` y por qué «cachear sí, contaminar no» no es contradictorio?

```
RFC 8020 — Negative DNS Responses and DNS Cache Pollution (2018)

QUÉ PIDE:
─────────

1. Cachear la respuesta negativa.  ✓ (protege contra la repetición)
2. Tratarla como un ERROR con nombre, NO como un nombre válido.
3. Guardar la entrada bajo la CLAVE DEL NOMBRE EXACTO que falló.

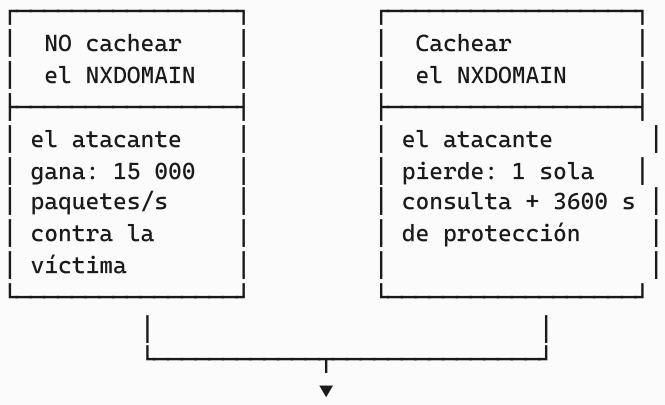
QUÉ PROHÍBE (el *domain shadowing*):
────────────────────────────────────

Que el NXDOMAIN de un nombre se derrame sobre otros nombres.

victima.example.com  →  NXDOMAIN  →  se guarda SÓLO para ese nombre
otra.example.com     →  ?         →  NO puede responder desde ese NXDOMAIN:
                                       tiene que consultarlo por separado

Y por qué esto importa en la práctica: si el NXDOMAIN se guardara
«en general» para el dominio, bastaría con que un nombre fallara una
vez para que TODOS los nombres del dominio quedaran «sombreados»
(shadowed), incluidos los que existen. Eso sería un DoS perfecto y
silencioso, hecho con cero paquetes.
```

La paradoja y su resolución:




```
La respuesta NO es «no cachear», es:
«cachear, pero con la LLAVE del nombre exacto,
 y sin dejar que el error se propague a los hermanos».

CACHEAR  = SÍ     (por nombre exacto)
CONTAMINAR = NUNCA (de un nombre a otro)
```

Interpretación física: la caché negativa es una **bomba de tiempo con reloj propio**: dura lo que diga el TTL del `SOA` (o el `minimum`), y después el nombre vuelve a ser consultable. Cachear de más es lo peligroso (permanece el error cuando el nombre ya existe: *domain shadowing*); cachear de menos es lo inútil (el atacante gana).

⚠️ Errores típicos

- **Decir que el `minimum` del `SOA` es «cuánto dura el NXDOMAIN» sin matizar.** El `minimum` es el **valor por defecto** para el TTL del negativo; si el autoritativo manda un `SOA` con otro TTL, **manda ese**.
- **Confundir `NXDOMAIN` con «el tipo no existe».** `NXDOMAIN` = **el nombre** no existe. Si el nombre existe pero no tiene ese tipo, es `NOERROR` con `ANCOUNT = 0` (NODATA).
- **Pensar que la caché negativa protege contra un atacante con nombres aleatorios.** No: protege contra la repetición. Contra la inundación de la tabla hace falta *rate limiting*.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-07, Pregunta 3 (8 puntos) — resolución iterativa sin caché

> **Fuente:** Examen `eirc1007` / `sirc1007` — 27 de julio de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1007.pdf

Enunciado (texto literal del examen)

> **Pregunta 3 (8 puntos)**
>
> **a)** Represente en un diagrama todos los pasos involucrados en la resolución de nombres iterativa donde el equipo necesito.laip.com.uy consulta a su servidor DNS local (dns.laip.com.uy) por la dirección IP asociada a www.fakaofo.tk. Suponga que no se dispone de ninguna información de caché en ningún servidor.
>
> **b)** Indique en el diagrama de la parte a) cuál es el servidor del TLD tk y cuál el autoritativo de fakaofo.tk.

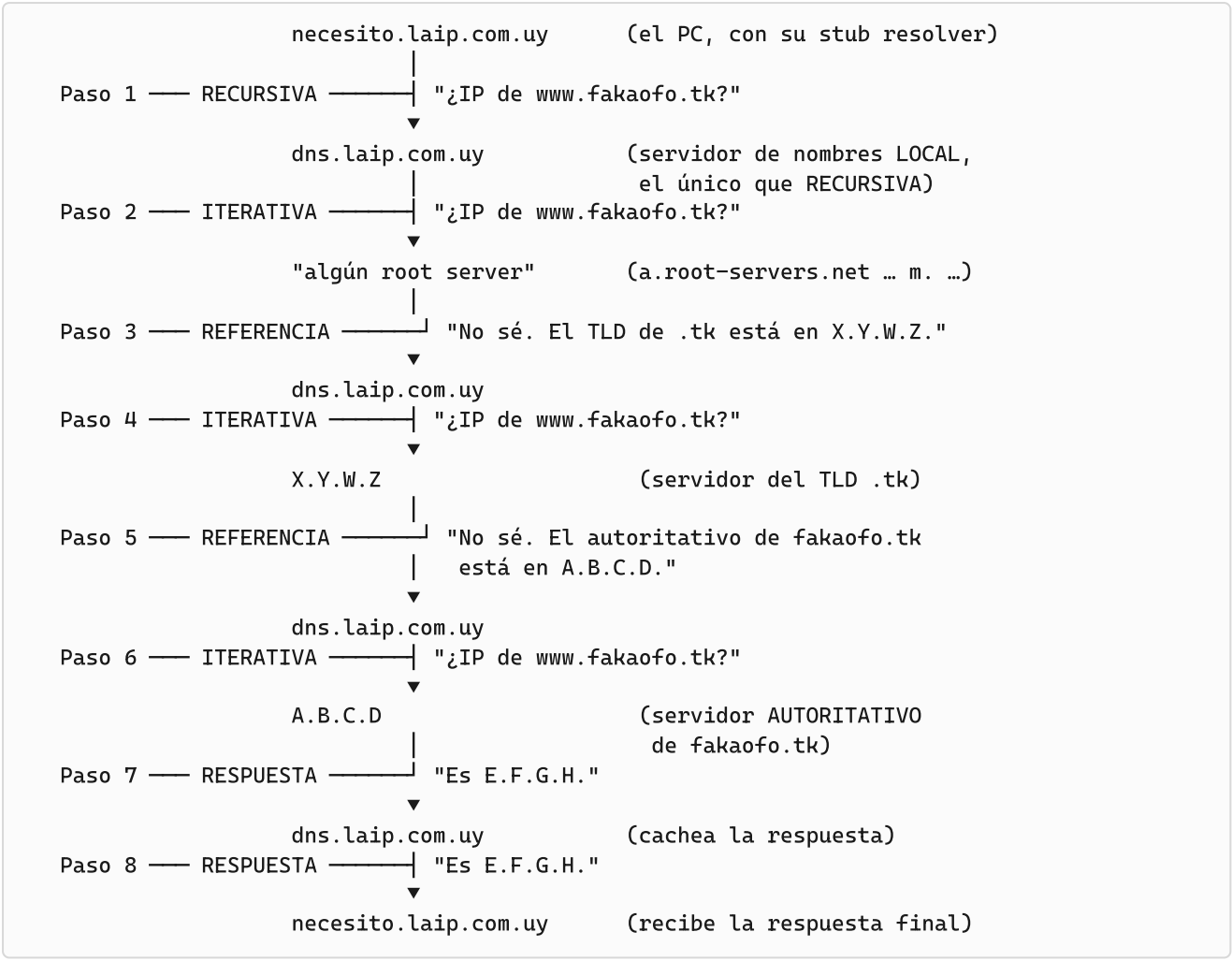
Solución oficial (del PDF, textual)

Paso	A	Información
1	necesito.laip.com.uy	dns.laip.com.uy
2	dns.laip.com.uy	"algún root server"
3	"algún root server" (el que recibió la consulta del paso 2)	dns.laip.com.uy
4	dns.laip.com.uy	X.Y.W.Z
5	X.Y.W.Z	dns.laip.com.uy
6	dns.laip.com.uy	A.B.C.D
7	A.B.C.D	dns.laip.com.uy
8	dns.laip.com.uy	necesito.laip.com.uy

> El diagrama debe ser muy similar al que figura en la diapo 9 de la clase 7 de capa de Aplicación. X.Y.W.Z, A.B.C.D y E.F.G.H son meramente ilustrativos, pero sí deben ser distintos.

a) Resolución — el diagrama completo

Acá está el diagrama pedido, con los cuatro actores y los ocho mensajes. Los nombres son los del examen:



Análisis de la solución oficial, punto por punto. Esto es lo que hay que aprender de ella:

✅ **Lo que está bien:**

1. **Los ocho mensajes están completos.** No falta ninguno: ni el 2 (al root), ni el 4 (al TLD), ni el 6 (al autoritativo), ni el 8 (la vuelta al PC). Es la estructura completa de §1.3, y coincide con la que da la cátedra.
2. **Los pasos 3 y 5 están bien escritos como *referencias*, no como respuestas.** La oficial dice «Consultar a TLD de tk», «Consultar a dns.fakaofo.tk», que es exactamente la definición de iterativa. Si en alguno de esos dos pasos hubiera puesto «la IP es …», la solución estaría mal.
3. **X.Y.W.Z, A.B.C.D y E.F.G.H son distintos**, como pide el enunciado: eso garantiza que cada paso es una consulta a **otra** máquina y no un invento.
4. **El paso 8 existe**, o sea que la respuesta final vuelve al PC. Muchos alumnos lo olvidan y se quedan en 7 pasos.

⚠️ **Lo que la oficial NO dice y hay que agregar para los 8 puntos:**

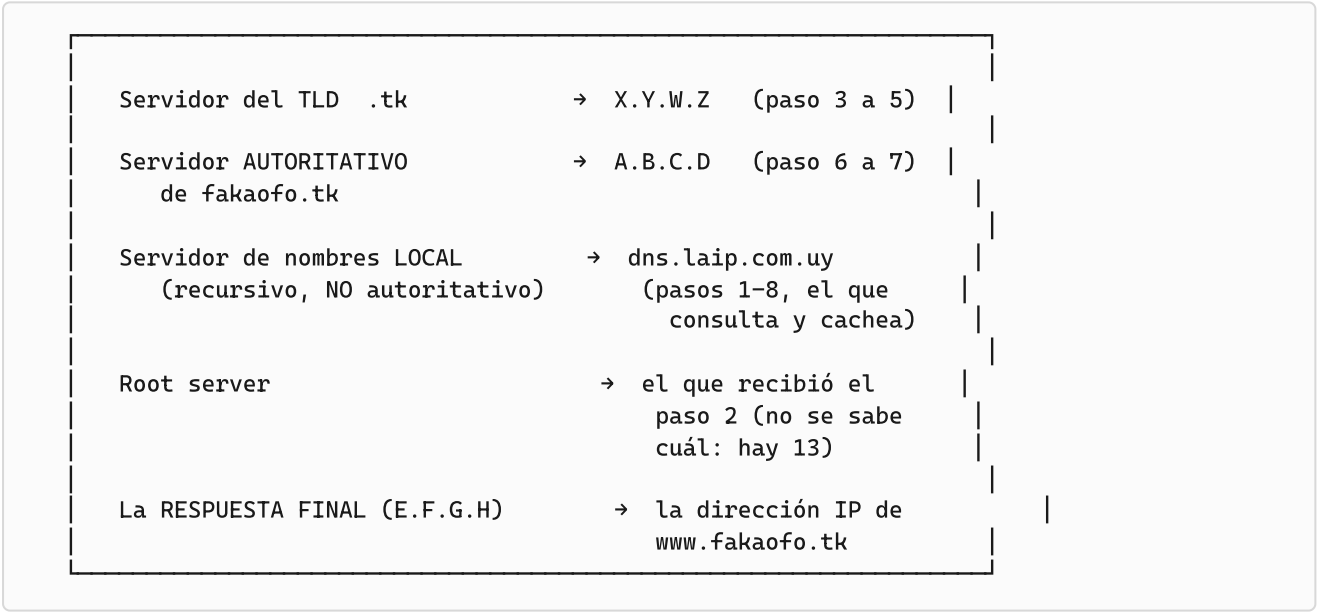
1. **No dice qué tipo de consulta es cada paso.** La oficial llama «Query» a los pasos 1, 2, 4 y 6 y «Response» a los 3, 5, 7 y 8. Eso alcanza, pero **suma** distinguir que los **pasos 3 y 5 son respuestas con** `ANCOUNT = 0`, es decir *referencias*, y que **solo el 7 tiene** `ANCOUNT = 1` **y** `AA = 1`.
2. **No dice qué algoritmo es cada paso.** El enunciado dice «resolución iterativa» y la oficial lo resuelve como iterativo en los pasos 2 a 7. Pero el **paso 1 es recursivo** en la realidad, y decir eso **suma puntos**

(ver la nota de abajo).

3. **No dice qué se cachea.** El enunciado dice «sin caché en ningún servidor», así que la oficial tiene razón en no tocarlo, pero **alumnos que mencionen qué se cachearía después** demuestran que entendieron.

> **Nota explícita contra la oficial — y es la más importante de esta clase:** el enunciado dice «**resolución de nombres iterativa**» y describe que `necesito.laip.com.uy` **consulta a su servidor DNS local**. En el modelo real, **esa primera consulta es recursiva**: el *stub resolver* del SO le pide al `dns.laip.com.uy` «resolveste esto por mí y traeme la respuesta final», y el flag `RD = 1` de la petición lo dice. Lo **iterativo** son los **cuatro pasos que van del servidor local hacia afuera** (2 a 7). Mi lectura: la oficial usa «iterativa» como abreviatura de «la que hace el servidor local», que es el nombre que se le da en la clase, y **no lo dice**. Si en un examen te piden el diagrama, **hacé los ocho pasos** (que es lo que se corrige) y **agregá la distinción de una línea**: «el paso 1 es recursivo y los 2 a 7 son iterativos». Eso no te resta nada y puede sumarte.

b) ¿Cuál es el TLD de `tk` y cuál el autoritativo de `fakaofo.tk`?



(solución propia: el PDF publica la tabla del apartado a) y remite a la diapo 9 de la clase 7 para el dibujo, pero no redacta la respuesta de b).)

El matiz que hay que agregar y es lo que suma puntos: `dns.laip.com.uy` **no es el autoritativo de nada**. Aunque sea el servidor de nombres de `laip.com.uy` (y puede serlo), frente a este problema está actuando como **resolver recursivo**: es un **cliente** de la jerarquía, no un **dueño** de datos. Su papel en el diagrama es el de un intermediario que consulta y cachea.

Respuesta redactada, lista para copiar en el examen:

> **a)** Con el equipo `necesito.laip.com.uy` y su servidor de nombres local `dns.laip.com.uy`, sin caché en ningún servidor, la resolución de `www.fakaofo.tk` son **ocho mensajes**:
>
> 1. `necesito.laip.com.uy` → `dns.laip.com.uy`: consulta por la IP de `www.fakaofo.tk`.

2. `dns.laip.com.uy` → un **root server**: la misma consulta.
3. El root **no conoce** `www.fakaofo.tk`; responde con una **referencia**: el servidor del TLD `.tk` está en `X.Y.W.Z`.
4. `dns.laip.com.uy` → `X.Y.W.Z` (TLD de `.tk`): la misma consulta.
5. El TLD tampoco conoce el dominio; responde con una **referencia**: el servidor autoritativo de `fakaofo.tk` es `dns.fakaofo.tk`, en `A.B.C.D`.
6. `dns.laip.com.uy` → `A.B.C.D` (autoritativo): la misma consulta.
7. El autoritativo **sí** conoce el mapeo y responde: `www.fakaofo.tk` es `E.F.G.H`.
8. `dns.laip.com.uy` → `necesito.laip.com.uy`: devuelve `E.F.G.H`.

Son **cuatro ida y vuelta = 4 RTT**: PC↔resolver (pasos 1 y 8), resolver↔root (2 y 3), resolver↔TLD (4 y 5) y resolver↔autoritativo (6 y 7). Los pasos 3 y 5 son **referencias**, no respuestas: llegan con `ANCOUNT = 0` y el dato en las secciones *Authority* y *Additional* (los `NS` y sus *glue records*). Solo el paso 7 es una respuesta final, y es la única con el flag `AA = 1`.

b) El servidor del TLD `.tk` es **`X.Y.W.Z`** y el servidor autoritativo de `fakaofo.tk` es **`A.B.C.D`**. El `dns.laip.com.uy` es el servidor de nombres **local (recursivo)**: no es autoritativo de `fakaofo.tk` ni de ningún otro nombre ajeno; en este problema actúa como un cliente más de la jerarquía, que consulta y cachea.

3.2 ⭐ Examen 2010-12, Pregunta 1 (8 puntos) — el TTL y los pasos de una consulta tipo A

Fuente: Examen `eirc1012` / `sirc1012` — 17 de diciembre de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1012.pdf **Este es el otro problema central de la unidad.**

Enunciado (texto literal del examen)

Pregunta 1 (8 puntos)

El DNS es una base de datos distribuida y jerárquica, compuesta por varios tipos de servidores.

a) Cuando un servidor de nombres aprende un mapeo, lo cachea. ¿Cuál es el campo del registro de recurso (RR) que decide si se puede cachear o no? ¿Cuál es el servidor que define ese tiempo? ¿El valor que toma ese campo, a qué refiere y en que unidades está expresado?

b) Suponga que desde un PC de una empresa, se hace una consulta tipo A al servidor de nombres local por el nombre pruebaexmen.edu, ¿cuales son los pasos siguientes que debe realizar el servidor antes de dar la respuesta al usuario?

Solución oficial (del PDF, textual)

> **a)** El campo es el TTL, que refiere al tiempo restante por el cual ese RR es válido y por lo tanto se puede mantener en caché. El TTL está definido por el servidor autoritativo de ese RR, y es la cantidad de segundos para los que ese registro es válido. El servidor que lo mantiene en cache, deberá ir decrementando ese valor, hasta que cuando llega a cero, deja de ser válido y se quita del caché.
>
> **b)** PC => Consulta SERVIDOR LOCAL por TIPO A, nombre pruebaexmen.edu SERVIDOR LOCAL => Consulta ROOT SERVER (no lo tiene en caché) por el servidor TLD del .edu ROOT SERVER responde con la IP del servidor TLD del edu SERVIDOR LOCAL => Consulta TLD del .edu por el RR tipo A, nombre pruebaexamen.edu Servidor TDL responde con la dirección IP del nombre pruebaexamen.edu al SERVIDOR LOCAL SERVIDOR LOCAL cachea la respuesta y responde al PC con la dirección IP del nombre pruebaexamen.edu

a) El TTL — resolución

La respuesta oficial es corta y precisa. Lo que hay que hacer es *desarmarla*.

Pregunta	Respuesta oficial	Dónde está en el RR
¿Qué campo decide si se puede cachear?	el **TTL** (*time to live*, tiempo de vida)	el **cuarto** de los cinco campos
¿Qué servidor define ese tiempo?	el **servidor autoritativo** de ese RR	quien escribe la zona
¿A qué refiere su valor?	al **tiempo restante** por el cual ese RR **es válido** y puede estar en caché	contador decreciente
¿En qué unidades?	**segundos**, siempre	4 bytes

La estructura del RR, con el TTL marcado:

```
nombre              clase   tipo    TTL      datos
fing.edu.uy.        IN      A       3600     132.42.120.5
─────────────       ──────  ────    ────     ──────────
nombre              2 bytes  2 bytes 4 bytes  variable
                             1       2
                                  ↑
                       EL TTL: 4 bytes, en SEGUNDOS
```

El razonamiento detrás de cada respuesta, que es lo que hay que saber explicar:

1. **¿Por qué el TTL y no otro campo?** Porque los otros cuatro **no dependen del tiempo**: el nombre es el nombre, la clase es `IN`, el tipo es `A` y los datos son la IP. **Solo el TTL es una declaración sobre cuánto tiempo vale lo que dice el registro.** Y por eso es el único que puede «envejecer»: al pasar el tiempo, el contenido sigue siendo cierto como hecho histórico, pero deja de ser *garantizado* por el origen.
2. **¿Por qué el autoritativo y no otro?** Porque **el TTL es una declaración de política del dueño del dato**, no una decisión técnica de quien lo transporta. Solo el que administra la zona sabe **con qué**

frecuencia cambia su contenido. Si lo definiera el resolver, cada resolver tendría su propia cuenta y el sistema se desincronizaría.

3. **¿Por qué «tiempo restante» y no «fecha de vencimiento»?** Porque el TTL es un **contador**, no una fecha. No hay reloj en el protocolo: no se manda «vence el 12/10 a las 15:00», se manda «3600 segundos». Cada quien que lo guarda lo cuenta **hacia abajo** desde el momento en que lo recibió. Es lo que permite que dos resolvers en distintos puntos del mundo coincidan sin sincronizar relojes.
4. **¿Por qué segundos?** Porque es la **única unidad que se Define en el formato del mensaje**: el campo ocupa 4 bytes y se interpreta como un entero sin signo de 32 bits. Cualquier otra unidad requeriría un campo de unidades, que no existe.

Respuesta redactada, lista para copiar en el examen:

> **a)** El campo es el **TTL** (*time to live*), que es el **cuarto campo** del registro de recurso y ocupa **4 bytes**.
>
> **Quién lo define:** el **servidor autoritativo** de esa zona, porque es el dueño del dato y es el único que sabe con qué frecuencia cambia. El TTL es una **declaración de política** del dueño del nombre.
>
> **A qué refiere:** al **tiempo restante** durante el cual ese registro es **válido** y, por lo tanto, puede mantenerse en la caché. No es una fecha absoluta sino un **contador decreciente**: el resolver que lo guarda lo va descontando desde el momento en que lo recibió, y cuando llega a cero el registro deja de ser fresco y obliga a revalidar.
>
> **Unidades: segundos**, siempre, sin excepción. El valor es un entero sin signo de 32 bits, así que el máximo es 4 294 967 295 segundos (unos 136 años).
>
> **Ejemplo:** `www.fing.edu.uy. 3600 IN A 132.42.120.5` significa «este registro es válido durante 3600 segundos (1 hora) a partir de ahora». Guardado a las 10:00:00, queda fresco hasta las 11:00:00; a las 11:00:01 hay que revalidar.
>
> **Nota contra la oficial:** la oficial es **correcta y completa** para el apartado a). Lo único que le agregaría es (i) que el TTL es el **cuarto** campo de los **cinco**, y que mide **4 bytes**, porque el enunciado habla de «el campo del RR» y conviene situarlo; y (ii) el **máximo representable**, que es un dato que a veces se pregunta y sale del formato del mensaje.

b) Los pasos de una consulta tipo A — resolución

Acá hay que tener cuidado, porque la solución oficial tiene un error conceptual. Primero la resolución completa y después el análisis.

El nombre del problema: `pruebaexamen.edu`. Descomponemos:

```
pruebaexamen.edu.
──────────── ───
2° nivel (2LD)   | TLD  (.edu es un gTLD "genérico",
= "pruebaexamen"  |  no es el ccTLD de un país)
                  ▼
raíz  →  .edu  →  pruebaexamen.edu  →  (el nombre buscado)
```

El recorrido completo, con los mismos ocho mensajes que en `2010-07 P3`:

#	De	A	Qué pasa
1	PC (stub)	servidor local (el de la empresa)	RECURSIVA: "¿IP de pruebaexamen.edu?" RD=1
2	servidor local	root server	ITERATIVA: "¿IP de pruebaexamen.edu?"
3	root server	servidor local	REFERENCIA: "el TLD .edu está en <IP del .edu>"
4	servidor local	TLD .edu	ITERATIVA: la pregunta
5	TLD .edu	servidor local	REFERENCIA: "el autoritativo es ns1.pruebaexamen.edu, en <IP del autoritativo>"
6	servidor local	autoritativo	ITERATIVA: la pregunta
7	autoritativo	servidor local	RESPUESTA: "es <IP>"
8	servidor local	PC	RESPUESTA: "es <IP>"

RTT: 2 (PC↔local) + 1 (local↔root) + 1 (local↔.edu) + 1 (local↔aut.) = 4 RTT
→ 8 mensajes.

El paso 9, que el enunciado pide explícitamente y que es el que da el punto:

```
9. El servidor local ARMA LA RESPUESTA y se la entrega al PC.

   ⚠️ Y ACA ESTÁ EL ERROR DE LA OFICIAL: el texto dice
   "Servidor TDL responde con la dirección IP del nombre
    pruebaexamen.edu al SERVIDOR LOCAL".

   El TLD .edu NO PUEDE responder con la dirección IP. No la tiene.
   El TLD .edu lo único que tiene de pruebaexamen.edu es el REGISTRO
   NS que dice quién lo autoriza. Si el TLD devolviera la IP, el
   sistema entero colapsaría en el TLD y no habría DNS distribuido.

   El TLD responde con una REFERENCIA (ANCOUNT=0), y la respuesta
   final la da el AUTORITATIVO en el paso 7.
```

Lo que hace el servidor local antes de responder (los 3_CACHEAR):

```
1. Cachea el REGISTRO A que vino del autoritativo,
   con el TTL que trae (que es el que definió el autoritativo).

2. Cachea la REFERENCIA del TLD .edu: el NS de pruebaexamen.edu
   y su glue (IP de ns1.pruebaexamen.edu), con el TTL que trae
   el TLD (típicamente 172800 s = 2 días).

   └── esto es lo que hace que la SEGUNDA resolución del mismo
       nombre tarde solo 2 RTT y no 4.

3. Cachea la REFERENCIA del root: el NS de .edu y su glue, con
   su TTL. Esta se cachea mucho menos porque el root tiene muchísimos
   TLD y la rotación es alta.

y recién después le responde al PC.
```

Respuesta redactada, lista para copiar en el examen:

b) Desde el PC de la empresa se hace una consulta **tipo A** al servidor de nombres local por `pruebaexamen.edu`. El nombre se resuelve **de derecha a izquierda**, y son estos los pasos:

1. El PC envía la consulta al **servidor de nombres local** de la empresa («¿qué dirección IPv4 corresponde a `pruebaexamen.edu`?»). Es una consulta **recursiva**: el stub resolver del sistema operativo le pide al servidor local que resuelva por él.
2. El servidor local, **sin tener el dato en caché**, hace una consulta **iterativa** a un **root server** por el mismo nombre.
3. El root server **no conoce** `pruebaexamen.edu`: no conoce ningún dominio, solo los TLD. Responde con una **referencia**: el servidor del TLD `.edu` está en determinada dirección IP.
4. El servidor local consulta **iterativamente** al **servidor del TLD** `.edu`.
5. El TLD `.edu` **tampoco conoce** el dominio. Responde con una **referencia**: el servidor **autoritativo** de `pruebaexamen.edu` es `ns1.pruebaexamen.edu`, cuya dirección IP viene en la sección *Additional* (el *glue record*).
6. El servidor local consulta **iterativamente** al **servidor autoritativo** de `pruebaexamen.edu`.
7. El autoritativo **sí** conoce el mapeo y responde con el **registro A** de `pruebaexamen.edu`, con su **TTL**.
8. El servidor local **cachea** la respuesta (el `A` con su TTL, y las referencias del TLD y del root con los suyos) y **le responde al PC** con la dirección IP.

Son **ocho mensajes** y **cuatro RTT**. Los pasos 3 y 5 son **referencias**, no respuestas (llegan con `ANCOUNT = 0`); la **única respuesta final** es la del paso 7, que es la única con el flag `AA = 1`.

Nota explícita contra la oficial — y es una diferencia real: La solución oficial describe **cinco pasos** y en el paso 5 dice que *«Servidor TDL responde con la dirección IP del nombre pruebaexamen.edu al SERVIDOR LOCAL»*. **Mi lectura es que ahí hay un error conceptual:** el servidor del TLD **no puede** responder con la dirección IP, porque **no la tiene y no puede tenerla** — si la tuviera, no habría DNS distribuido, todo el mundo dependería de los TLD y el sistema no escalaría. Lo que el TLD devuelve es el **registro** `NS` que lo delega, es decir, **quién es el autoritativo** (más su *glue*), y **a partir de ahí hay un paso más**: la consulta al servidor autoritativo, que es el único que devuelve el registro `A`. Por eso la cadena completa son **ocho mensajes (cuatro RTT)** y no cinco: es exactamente la misma cadena del `2010-07 P3`. Si el enunciado acepta cinco pasos porque es lo que se da en la clase, **reproduce la tabla oficial** y **agregá una línea con el paso que falta**: eso no te resta nada y muestra que entendiste la jerarquía.

3.2.1 Los otros tres puntos de la misma pregunta de 2010-12 (por completitud)

La Pregunta 1 del examen de diciembre de 2010 vale 8 puntos y la cátedra la suele usar **entera** como pregunta de concepto. Los otros tres son de otros temas, pero conviene tenerlos frescos porque se repreguntan:

Apartado	En una línea
P2a.1 ¿Cómo deriva un segmento UDP a su proceso?	por el **puerto destino**
P2a.2 ¿Cómo sabe el proceso a dónde mandar la respuesta?	por **IP origen + puerto origen**
P2a.3 ¿Qué identifica un socket UDP en la red?	**IP del host + número de puerto** (la «2-tupla», simplificación clásica)
P2b ¿Y un socket TCP?	la **4-tupla**: IP origen, puerto origen, IP destino, puerto destino

(Se desarrolla entero en la Clase 16, «UDP: multiplexación».)

3.3 Examen 2007-07, Pregunta 4 (8 puntos) — función del DNS y protocolo involucrado

> **Fuente:** Examen `eirc0707` / `sirc0707` — julio de 2007.
>
> **Enunciado:** Describa la función del sistema de nombres de dominio y qué protocolo se utiliza para la resolución de nombres.

Resolución (síntesis de §1.1, §1.2 y §1.7 de esta clase y de la Clase 10).

a) La función: el DNS es un **espacio de nombres jerárquico** y una **base de datos distribuida** que traduce **nombres de host a direcciones IP** y viceversa (`PTR`). Es **jerárquico** (raíz → TLD → autoritativo, resuelto de derecha a izquierda) y **distribuido** (cada servidor solo conoce los datos de su zona y delega el resto), lo que le da escala, tolerancia a fallas y mantenimiento distribuido de la autoridad.

b) El protocolo: un protocolo de **capa de aplicación** que viaja sobre **UDP puerto 53** en el caso normal, y sobre **TCP puerto 53** cuando la respuesta no cabe en un datagrama (banderita `TC`) o cuando se pide una **transferencia de zona** (`AXFR` / `IXFR`). El mensaje tiene **12 bytes de cabecera** (ID, *flags*, y los contadores `QDCOUNT` / `ANCOUNT` / `NSCOUNT` / `ARCOUNT`) más cuatro secciones: **Question**, **Answer**, **Authority** y **Additional**.

Lo que hay que agregar y suma puntos: el protocolo usa **consultas iterativas y recursivas**, y el flujo real es que el **stub resolver del host hace una consulta recursiva al resolver local**, y el resolver local hace **consultas iterativas** al root, al TLD y al autoritativo. **Y con DNSSEC (`RFC 4033–4035`, 2005) el mismo protocolo transporta además firmas digitales**, aunque el despliegue de esa capa es parcial: hoy están firmados ~99,6 % de los gTLD pero solo ~5-7 % de los dominios.

PARTE 4 — CIERRE

📌 Resumen

1. **Iterativa**: el que no sabe devuelve una **referencia** («preguntale a este») y no hace nada más; **quien sigue preguntando es el cliente**. **Recursiva**: el que no sabe **pregunta él** al nivel inferior; **quien sigue preguntando es el servidor**.
2. **El flujo real**: el *stub resolver* del SO → **recursiva** → resolver local; el resolver local → **iterativas** → root → TLD → autoritativo. **No cambió desde 1987.**
3. **Son 8 mensajes y 4 RTT** sin caché. Los mensajes 3 y 5 son **referencias** (`ANCOUNT = 0`, dato en *Authority* + *Additional* con los *glue*); solo el 7 es respuesta final (`AA = 1`).
4. **La caché existe** porque sin ella cada visita a una página generaría 2 a 6 consultas, y los 13 root no darían abasto. **Ahorra viajes, no bytes.**
5. **El TTL es el 4.º campo del RR, de 4 bytes, en segundos.** Lo define el **servidor autoritativo**; lo **descuenta** el resolver que lo cacheó, y cuando llega a cero la entrada deja de ser fresca y hay que **revalidar**.
6. **El TTL es un compromiso**: corto → los cambios se propagan rápido pero el origen recibe muchas consultas; largo → el origen recibe pocas consultas, pero los cambios tardan hasta un TTL en propagarse.
7. **La caché negativa** cachea el `NXDOMAIN` con el TTL del `SOA` (o el `SOA.MINIMUM`), y eso **protege** contra ataques de denegación por repetición. `RFC 8020` dice: cachear **sí**, pero **bajo la clave del nombre exacto**, sin contaminar (*domain shadowing*).
8. ⚠️ **Actualización a 2026: DoH (`RFC 8484`)** ya es el cambio mayor —el DNS del navegador **ya no lo ve tu ISP**—; **Chrome no cambia tu resolver**, intenta «subir» el que ya tenías a DoH si está en su lista de proveedores conocidos, con *fallback* a DNS plano. **DoT (`RFC 7858`, 853)** y **DoQ (`RFC 9250`)** van por UDP/TCP 853. `RFC 8767` permite servir caché vencida. **`SVCB` / `HTTPS` (`RFC 9460`)** hace que el DNS transporte política de transporte. **DNSSEC** (`RFC 4033–4035`): ~99,6 % de los gTLD firmados, solo **~5-7 %** de los dominios: **estandarizado en 2005, despliegue bajo veinte años después.**
9. **Puerto 53 UDP y TCP**, y **853** para DoT. DoH usa el **443**, que es justamente por lo que se desplegó más: si podés abrir una web, podés resolver un nombre.
10. **Si te preguntan «qué cambió en el DNS»**, la respuesta es **nada de la arquitectura**. Lo que se agregó son cifrado, autenticidad parcial, extensiones, una semántica de caché resiliente y política de transporte.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Cuál es la diferencia entre una consulta iterativa y una recursiva? En una línea cada una.

▶ Respuesta

2. En el flujo real, ¿qué consulta es recursiva y cuáles son iterativas?

▶ Respuesta

3. ¿Cuántos mensajes y cuántos RTT tiene una resolución sin caché? ¿Cuáles de esos mensajes son respuestas y cuáles son referencias?

▶ Respuesta

4. ¿Cuál es el campo del RR que decide si se puede cachear? ¿Quién lo define? ¿Qué unidades tiene?

▶ Respuesta

5. ¿Qué es la caché negativa y por qué es una **protección** y no un problema?

▶ Respuesta

6. ¿Qué es DoH y por qué «Chrome usa el DNS de Google por defecto» es falso?

▶ Respuesta

7. El TTL es 3600 y el registro se guardó a las 10:00:00. ¿Cuándo deja de ser fresco? ¿Y cuándo se borra de la caché?

▶ Respuesta

8. Un PC resuelve `www.fing.edu.uy`: 2 ms hasta su resolver, 40 ms al root, 25 ms al TLD y 15 ms al autoritativo. ¿Cuánto tarda (a) sin ninguna caché, (b) con el `NS` de `fing.edu.uy` cacheado, (c) con el `A` cacheado y fresco?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1007` / `sirc1007`** — Pregunta 3 (8 pts): ★ resolución iterativa completa de `www.fakaofo.tk` sin caché, y qué servidor es el TLD y cuál el autoritativo
- **Examen `eirc1012` / `sirc1012`** — Pregunta 1 (8 pts): ★ (a) el TTL del RR, quién lo define, a qué refiere y unidades; (b) los pasos de una consulta tipo A a `pruebaexamen.edu`
- **Examen `eirc1002` / `sirc1002`** — Pregunta 1b (8 pts): iterativas vs. recursivas
- **Examen `eirc0707` / `sirc0707`** — Pregunta 4 (8 pts): función del DNS y protocolo involucrado
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 7 §7.4 («El sistema de nombres de dominio»), §7.4.2 (espacio de nombres jerárquico) y §7.4.3 (resolución de nombres)
- **RFC 1034** — *Domain Names: Concepts and Facilities* (1987): el diseño completo, que **no cambió** · **RFC 1035** — *Implementation and Specification* (1987): el formato del mensaje y de los RR
- **RFC 8484** — DNS over HTTPS, DoH (2018) · **RFC 7858** — DNS over TLS, DoT (2016, puerto 853) · **RFC 9250** — DNS over QUIC, DoQ (2022, sin *head-of-line blocking*)

- **RFC 8020** — *Negative DNS Responses and DNS Cache Pollution* (2018): `NXDOMAIN` se cachea, pero por nombre exacto
- **RFC 8767** — servir caché vencida cuando el autoritativo no responde (2020)
- **RFC 6891** — EDNS(0) (2013, *Internet Standard*): el pseudo-registro `OPT` · **RFC 7766** — conexiones TCP de DNS persistentes y reutilizables (2016)
- **RFC 9460** — SVCB/HTTPS (2023): el DNS transporta política de transporte y descubre HTTP/3 antes de conectar · **RFC 9462** — DDR, descubrir resolvers en `_dns.resolver.arpa`
- **RFC 4033-4035** — DNSSEC (2005) · **RFC 5155** — raíz firmada (2010) · **RFC 8080** — EdDSA · **RFC 8901** — firmantes múltiples
- **IANA Service Name and Transport Protocol Port Number Registry** — 53/udp, 53/tcp, 853/tcp (DoT)
- **Clase anterior:** 10 — DNS: la base de datos distribuida
- **Clase siguiente:** 12 — El resolver DNS en la práctica, con sockets